

Создание скрипта инициализации для BusyBox

Владислав Богданов

2026-06-08

Оглавление

- 1. Введение 1
- 2. Создай файл скрипта 1
- 3. Вставь туда следующий код: 1
- 4. Сделай скрипт исполняемым: 2
- 5. Промежуточный итог 2
- 6. А что если есть пользовательское ПО 2

1. Введение

Если ядро не позволяет менять Device Tree на лету, мы говорим системе: "Сделай это через 2 секунды после загрузки, пока пользователь еще даже не успел залогиниться".

В Linux (и в BusyBox в частности) за автозагрузку отвечают скрипты в `/etc/init.d/`.

2. Создай файл скрипта

```
busybox vi /etc/init.d/S99pinmux-fix
```

В Linux префикс S99 используется для автозапуска скриптов и сервисов на последнем этапе загрузки операционной системы в классической системе инициализации System V (SysVinit).

Структура имени означает следующее:



- S (Start) — указывает демону инициализации, что скрипт нужно запустить (в отличие от префикса K, который означает Kill — остановить сервис).
- 99 — приоритет выполнения от 00 до 99. Число 99 является максимальным, поэтому скрипты с этим префиксом запускаются самыми последними, когда система уже полностью загружена и сеть активна.

Где это применяется?

Файлы с такими префиксами хранятся в каталогах уровней выполнения: `/etc/rc.d/rcX.d/` или `/etc/rcX.d/` (где X — это номер уровня, например 2, 3 или 5). В современной практике администраторы создают символические ссылки вида `S99myscript` → `/etc/init.d/myscript`, чтобы собственный скрипт запускался автоматически при старте.

3. Вставь туда следующий код:

```
#!/bin/sh
# S99pinmux-fix: Переключение PA19 с UART1_RTS на GPIO

case "$1" in
  start)
    echo "Applying PA19 pinmux fix..."
    # 1. Переключаем мультимплексор (3 = GPIO для PA19)
    devmem 0x03001064 32 3

    # 2. Экспортируем и настраиваем GPIO (на всякий случай)
```

```

echo 499 > /sys/class/gpio/export 2>/dev/null
echo out > /sys/class/gpio/gpio499/direction
echo 0 > /sys/class/gpio/gpio499/value
;;
stop)
# При выключении можно вернуть как было, если нужно
devmem 0x03001064 32 4
;;
restart)
$0 stop
$0 start
;;
*)
echo "Usage: $0 {start|stop|restart}"
exit 1
;;
esac
exit 0

```

4. Сделай скрипт исполняемым:

```
chmod +x /etc/init.d/S99pinmux-fix
```

5. Промежуточный итог

Теперь при каждой перезагрузке платы пин будет автоматически становиться GPIO. Это абсолютно легитимный и широко используемый метод в индустрии, когда вендор предоставил образ в котором необходимы изменения, а пересобирать весь образ лень.

6. А что если есть пользовательское ПО

Если помимо перенастройки нужно запустить пользовательское ПО, то поступаем аналогичным образом. Но есть **три критически важных правила**, чтобы твое приложение не "повесило" загрузку платы.

Три золотых правила запуска приложений на старте

1. Абсолютные пути. В момент загрузки переменная \$PATH еще не настроена. Нельзя написать просто myapp. Нужно писать /usr/bin/myapp или /root/myapp.
2. Фоновый режим (Backgrounding). Если программа содержит бесконечный цикл (while(1)), она заблокирует процесс загрузки. Система будет ждать её завершения, которого не наступит. Нужно запускать её в фоне через &.
3. Перехват вывода. Если приложение пишет в stdout (через printf), этот текст может попасть в консоль и испортить логи загрузки. Лучше перенаправлять вывод в файл или в /dev/null.

Шаблон идеального скрипта запуска

Этот скрипт, который сначала делает наш трюк с devmem (настройка железа), а затем запускает программу.

Создадим файл /etc/init.d/S99myapp:

```
#!/bin/sh

# Путь к твоему приложению (замени на реальный)
APP_PATH="/usr/bin/my_super_app"
# Файл для логов
LOG_FILE="/var/log/myapp.log"
# Файл для хранения PID (чтобы потом можно было убить процесс)
PID_FILE="/var/run/myapp.pid"

case "$1" in
  start)
    echo "Starting my_super_app..."

    # 1. Сначала настраиваем железо (наш фикс для PA19)
    devmem 0x03001064 32 3
    echo 499 > /sys/class/gpio/export 2>/dev/null
    echo out > /sys/class/gpio/gpio499/direction
    echo 0 > /sys/class/gpio/gpio499/value

    # 2. Проверяем, не запущено ли приложение уже
    if [ -f "$PID_FILE" ]; then
      echo "App is already running!"
      exit 1
    fi

    # 3. Запускаем приложение в фоне (&) и перенаправляем вывод
    $APP_PATH > $LOG_FILE 2>&1 &

    # 4. Сохраняем PID запущенного процесса
    echo $! > $PID_FILE
    echo "App started with PID $(cat $PID_FILE)"
    ;;

  stop)
    echo "Stopping my_super_app..."
    if [ -f "$PID_FILE" ]; then
      kill $(cat $PID_FILE)
      rm -f $PID_FILE
      echo "App stopped."
    else
      echo "App is not running."
    fi
    ;;
endcase
```

```
restart)
    $0 stop
    sleep 1
    $0 start
    ;;

*)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
    ;;
esac

exit 0
```

Как это использовать:

1. Сохрани этот код в файл `/etc/init.d/S99myapp`.
2. Сделай его исполняемым:

```
chmod +x /etc/init.d/S99myapp
```

1. Положи свой скомпилированный бинарник в `/usr/bin/my_super_app` (и сделай его исполняемым: `chmod +x /usr/bin/my_super_app`).
2. Перезагрузи плату

```
reboot
```

После загрузки плата сама настроит пин, запустит твоё приложение в фоне, а все `printf` из твоей программы будут аккуратно складываться в `/var/log/myapp.log`. Если тебе понадобится остановить приложение вручную, ты сможешь сделать это командой:

```
/etc/init.d/S99myapp stop
```

Нюанс для C/C++ разработчиков



Если ты пишешь приложение на C/C++, которое должно работать как демон (служить вечно в фоне), хорошей практикой считается делать "демонизацию" внутри самого кода (через системный вызов `daemon(0, 0)` или двойной `fork()`), а не просто добавлять `&` в скрипт. Но для начала (и для 90% простых embedded-задач) запуск через `&` в `init`-скрипте — это абсолютно нормальный и рабочий подход.